SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	HARIHARAN.R	**Reg. No.:**	192524003
Date:	08.10.2026	**Duration:**	60 minutes

Code of Conduct

I _____HARIHARAN.R(192524003)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARIHARAN.R_____

Annexure A – Case Study Analysis

Machine Learning Model Drift

Scenario

A fraud detection system at a payment processor was highly accurate at launch but its precision has degraded significantly over 18 months as fraudsters adapted their techniques.

Questions

a) How would you design an MLOps pipeline to detect and address model drift continuously?

b) Discuss monitoring strategies, retraining triggers, shadow deployment, and A/B testing of models.

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.

- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
   - Justify why the selected methodology/framework is suitable for the proposed system.
   - Explain how it supports successful project development and maintenance.
6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering	Proposes an innovative, feasible solution applying appropriate software engineering	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.

Principles (15%)	principles (modularity, scalability, maintainability, security, etc.).			
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

ANSWER::

Machine Learning Model Drift
Introduction
Machine Learning models are widely used in payment processing systems to detect fraudulent transactions. A fraud detection model is trained using historical transaction data and learns patterns that help identify whether a transaction is genuine or fraudulent. However, fraudsters continuously change their methods, transaction patterns, and attack techniques. As a result, the data distribution and fraud patterns may change over time, causing the model's performance to decrease. This phenomenon is known as Machine Learning Model Drift.
Model drift is particularly important in fraud detection because a model that performs well during deployment may become less accurate after several months. Therefore, an effective MLOps (Machine Learning Operations) pipeline is required to continuously monitor the model, detect drift, retrain it with recent data, and safely deploy improved models.

a) Design an MLOps Pipeline to Detect and Address Model Drift Continuously
An MLOps pipeline for fraud detection should automate the complete process from data collection to monitoring and model retraining.

1. Data Collection
The first step is to continuously collect transaction data from the payment processing system.
The data may contain:
Transaction amount
Transaction location
Time and date
Payment method
Device information
Customer behaviour
Fraud/legitimate transaction label
Both recent transaction data and historical data should be stored for analysis.

2. Data Validation and Preprocessing
Before using the data for training, the pipeline should check the quality of the incoming data.
It should detect:
Missing values
Duplicate transactions
Incorrect data types
Unexpected values

Changes in feature distributions
After validation, the data is cleaned and transformed into the format required by the ML model.

3. Model Training
A fraud detection algorithm such as Logistic Regression, Random Forest, XGBoost, or Neural Network can be trained using historical transaction data.
The training pipeline should be automated so that a new model can be created whenever sufficient new data becomes available.

4. Model Performance Monitoring
The deployed model should be continuously monitored using metrics such as:
Precision
Recall
F1-score
Accuracy
False Positive Rate
False Negative Rate
ROC-AUC
For fraud detection, precision and recall are especially important because incorrectly identifying genuine transactions as fraud can affect customers, while missing fraudulent transactions can cause financial losses.

5. Data Drift Detection
The system should compare the distribution of current transaction data with the training data.
For example, if the model was trained when most fraudulent transactions came from one type of device, but fraudsters later start using different devices, the feature distribution changes.
Statistical methods such as PSI (Population Stability Index), KL divergence, and statistical hypothesis tests can be used to detect this change.

6. Model Drift Detection
Model drift occurs when the relationship between input data and the target variable changes.
For example:
Before:
Fraudulent transaction → unusual location + high amount
After 18 months:
Fraudulent transaction → normal location + multiple small transactions
The old model may fail to recognize the new fraud pattern.

When performance drops below a predefined threshold, the pipeline should generate an alert and initiate retraining.

7. Automated Retraining
Newly collected and labelled transaction data should be combined with historical data to train an updated model.
A possible pipeline is:
New Data → Validation → Drift Detection → Retraining → Model Evaluation → Deployment
The new model should only be deployed if it performs better than the existing production model.

8. Model Registry and Version Control
Every trained model should be stored with a version number.
For example:
Model V1 – Initial model
Model V2 – After 6 months
Model V3 – After fraud pattern changes
The model registry allows the organization to compare models and easily roll back to a previous stable model if required.

9. Safe Deployment
The new model should not immediately replace the existing model.
It can first be deployed using shadow deployment, followed by A/B testing, and finally full production deployment if the new model performs well.
Overall MLOps Pipeline
Transaction Data → Data Validation → Drift Monitoring → Model Training → Model Evaluation → Model Registry → Shadow Deployment → A/B Testing → Production → Continuous Monitoring → Retraining

b) Monitoring Strategies, Retraining Triggers, Shadow Deployment and A/B Testing
1. Monitoring Strategies
Continuous monitoring is essential because model performance can change after deployment.
a) Data Monitoring
Monitor changes in:
Transaction amount
Transaction frequency
Location
Device type
Payment method
Customer behaviour

b) Model Performance Monitoring
Monitor:
Precision
Recall
F1-score
False Positive Rate
False Negative Rate
ROC-AUC
For example, if precision decreases from 95% to 82%, an alert can be generated.

c) Business Monitoring
Technical metrics alone are not sufficient. Business metrics should also be monitored, such as:
Fraud loss
Number of blocked genuine transactions
Customer complaints
Number of successful fraud attempts
Financial loss prevented
This helps determine whether the model is actually providing business value.

2. Retraining Triggers

The model should be retrained when predefined conditions are reached.
Important retraining triggers include:
Trigger 1 – Performance Drop
If precision or recall falls below a predefined threshold.
Example:
If precision < 90%, start retraining.
Trigger 2 – Data Drift
If the distribution of important features changes significantly.

Example:
If PSI exceeds the organization's defined threshold, investigate and consider retraining.
Trigger 3 – Concept Drift
When fraudsters change their behaviour and the relationship between features and fraud changes.
For example, fraudsters may move from large transactions to many small transactions.
Trigger 4 – Time-Based Retraining
The model can also be retrained periodically.
For example:
Every month → Collect new data → Retrain → Evaluate → Deploy if better

A combination of automatic time-based retraining and performance-based triggers is usually more effective than relying on only one trigger.

3. Shadow Deployment
In shadow deployment, the new model receives the same live transaction data as the existing production model, but its predictions are not used to make real decisions.
For example:
Transaction → Old Model → Actual Decision
At the same time:
Transaction → New Model → Prediction only
The predictions of the new model are recorded and compared with the actual outcomes.
Advantages
No risk to customers
No impact on live transactions
Real-world performance can be evaluated
Helps identify unexpected problems before production deployment
For the fraud detection system, shadow deployment is useful because the new model can be tested against current fraud techniques without affecting actual payment decisions.

4. A/B Testing of Models
After successful shadow testing, the new model can be evaluated using A/B testing.
Two models are used:
Model A: Existing production model
Model B: New model
A small percentage of transactions can be processed using Model B, while the majority continue using Model A.

For example:
90% transactions → Model A
10% transactions → Model B
Their performance is compared using:
Precision
Recall
F1-score
False positive rate
Fraud detection rate
Business loss
If Model B consistently performs better without causing unacceptable customer impact, the percentage can gradually be increased.
10% → 25% → 50% → 100%If the new model performs poorly, the system can immediately roll back to Model A.

Conclusion

Machine Learning Model Drift is a major challenge in fraud detection because fraudsters continuously develop new techniques. A continuous MLOps pipeline can solve this problem by combining data monitoring, model performance monitoring, drift detection, automated retraining, model versioning, shadow deployment, and A/B testing. Continuous monitoring helps identify performance degradation early, while shadow deployment and A/B testing allow new models to be safely evaluated before complete deployment. Therefore, a well-designed MLOps system ensures that the fraud detection model remains accurate, reliable, adaptable, and effective over time.